ANA515 Project Part 2 - R Script

Anushree Das

For new changes added to the pipeline, jump to Advanced Dimensional Modeling

```
#Load necessary libraries
library(dplyr)
library(lubridate)
library(tidyverse)
library(ggplot2)
```

1. EXTRACT

```
# Load dataset from csv
users_csv  <- read.csv("credit_card_transactions/sd254_users.csv")
cards_csv <- read.csv("credit_card_transactions/sd254_cards.csv")
transactions_csv <- read.csv("credit_card_transactions/credit_card_transactions-ibm_v2.csv")
```

2. TRANFORM

2.1 Modify users table

2.1.a Generate UserId column for users table

2.1.b Rename column names

```
users <- data.frame(UserId = 1:nrow(users_csv),
                        FullName = users_csv$Person,
                        CurrentAge = users_csv$Current.Age,
                        RetirementAge = users_csv$Retirement.Age,
                        BirthYear = users_csv$Birth.Year,
                        BirthMonth = users_csv$Birth.Month,
                        Gender= users_csv$Gender,
                        Address1 = users_csv$Address,
                        Address2 = users_csv$Apartment,
                        City = users_csv$City,
                        State = users_csv$State,
                        Zipcode = users_csv$Zipcode,
                        Latitude = users_csv$Latitude,
                        Longitude = users_csv$Longitude,
                        PerCapitaIncomeZipcode = users_csv$Per.Capita.Income...Zipcode,
```

```r
                        YearlyIncome = users_csv$Yearly.Income...Person,
                        TotalDebt = users_csv$Total.Debt,
                        FICOScore = users_csv$FICO.Score,
                        NumCreditCards = users_csv$Num.Credit.Cards
)

head(users, 3)
```

```
##   UserId       FullName CurrentAge RetirementAge BirthYear BirthMonth Gender
## 1      1 Hazel Robinson         53            66      1966         11 Female
## 2      2     Sasha Sadr         53            68      1966         12 Female
## 3      3     Saanvi Lee         81            67      1938         11 Female
##                 Address1 Address2        City State Zipcode Latitude Longitude
## 1          462 Rose Lane       NA    La Verne    CA   91750    34.15   -117.76
## 2 3606 Federal Boulevard       NA Little Neck    NY   11363    40.76    -73.74
## 3        766 Third Drive       NA West Covina    CA   91792    34.02   -117.89
##   PerCapitaIncomeZipcode YearlyIncome TotalDebt FICOScore NumCreditCards
## 1                 $29278       $59696   $127613       787              5
## 2                 $37891       $77254   $191349       701              5
## 3                 $22681       $33483      $196       698              5
```

2.1.c Convert Gender to categorical data type

```r
users$Gender <- factor(users$Gender)
```

2.1.d Convert columns - Address2, Zipcode to Text data type

Zipcode and Address2 should be converted to Text data type because they are not used for quantitative statistics

```r
users$Address2 <- as.character(users$Address2)
users$Zipcode <- as.character(users$Zipcode)
```

2.1.e Convert columns - PerCapitaIncomeZipcode, YearlyIncome, TotalDebt to numeric

```r
# Remove $ character from the column values and convert to numeric data type
users$PerCapitaIncomeZipcode <- as.numeric(sub('.', '', users$PerCapitaIncomeZipcode))
users$YearlyIncome <- as.numeric(sub('.', '', users$YearlyIncome))
users$TotalDebt <- as.numeric(sub('.', '', users$TotalDebt))
```

2.2 Modify cards tables

2.2.a Remove duplicates

```r
cards_csv <- cards_csv %>% distinct()
```

2.2.b Combine user id and card index to generate CardId

2.2.c Rename column names

```
cards <- data.frame(CardId = paste(cards_csv$User, cards_csv$CARD.INDEX, sep = "$"),
                        CardBrand = cards_csv$Card.Brand,
                        CardType = cards_csv$Card.Type,
                        CardNumber = cards_csv$Card.Number,
                        Expires = cards_csv$Expires,
                        CVV = cards_csv$CVV,
                        HasChip = cards_csv$Has.Chip,
                        CardsIssued = cards_csv$Cards.Issued,
                        CreditLimit = cards_csv$Credit.Limit,
                        AcctOpenDate = cards_csv$Acct.Open.Date,
                        YearPINLastChanged = cards_csv$Year.PIN.last.Changed
)
head(cards, 3)
```

```
##   CardId CardBrand CardType   CardNumber Expires CVV HasChip CardsIssued
## 1    0$0      Visa    Debit 4.344677e+15 12/2022 623     YES           2
## 2    0$1      Visa    Debit 4.956966e+15 12/2020 393     YES           2
## 3    0$2      Visa    Debit 4.582313e+15 02/2024 719     YES           2
##   CreditLimit AcctOpenDate YearPINLastChanged
## 1      $24295      09/2002               2008
## 2      $21968      04/2014               2014
## 3      $46414      07/2003               2004
```

2.2.d Convert columns - CardBrand, CardType, HasChip to categorical data type

```
cards$CardBrand <- factor(cards$CardBrand)
cards$CardType <- factor(cards$CardType)
cards$HasChip <- factor(cards$HasChip)
```

2.2.e Convert columns - Expires, AcctOpenDate to Date data type

The Date Format for Expires and AcctOpenDate contains only Month and Year, but R needs a value for Day as well. The Expires date will be set to the last day of the month and AcctOpenDate to the first day of the month for this project.

```
cards$Expires <- my(cards$Expires)
cards$Expires <- ceiling_date(cards$Expires, "month") - days(1)

cards$AcctOpenDate <- my(cards$AcctOpenDate)
```

2.2.f Convert columns - CardNumber, CVV to Text data type

CardNumber and CVV not used for quantitative statistics, therefore they are being converted to Text

```r
cards$CardNumber <- as.character(cards$CardNumber)
cards$CVV <- as.character(cards$CVV)
```

2.2.g Convert column CreditLimit to numeric

```r
cards$CreditLimit <- as.numeric(sub('.', '', cards$CreditLimit))
```

2.3 Create table for merchants

2.3.a Extract Merchant related column from the Transactions table

```r
merchants_temp <- data.frame(MerchantName = transactions_csv$Merchant.Name,
                             MerchantCity = transactions_csv$Merchant.City,
                             MerchantState = transactions_csv$Merchant.State,
                             MerchantZIP = transactions_csv$Zip,
                             MerchantCategoryCode = transactions_csv$MCC
)
```

2.3.b Remove redundant data from the Merchants table

```r
nrow(merchants_temp)
```

```
## [1] 24386900
```

```r
length(unique(merchants_temp$MerchantName))
```

```
## [1] 100343
```

```r
merchants_temp <- merchants_temp %>% distinct(MerchantName, .keep_all = TRUE)
nrow(merchants_temp)
```

```
## [1] 100343
```

2.3.c Generate and add MerchantId column

```r
merchants <- data.frame(MerchantId = 1:nrow(merchants_temp),
                        MerchantName = merchants_temp$MerchantName,
                        MerchantCity = merchants_temp$MerchantCity,
                        MerchantState = merchants_temp$MerchantState,
                        MerchantZIP = merchants_temp$MerchantZIP,
                        MerchantCategoryCode = merchants_temp$MerchantCategoryCode
)

head(merchants, 3)
```

```
##   MerchantId  MerchantName  MerchantCity MerchantState MerchantZIP
## 1          1  3.527213e+18      La Verne            CA       91750
## 2          2 -7.276121e+17 Monterey Park            CA       91754
## 3          3  3.414527e+18 Monterey Park            CA       91754
##   MerchantCategoryCode
## 1                 5300
## 2                 5411
## 3                 5651
```

2.3.d Convert columns - MerchantName, MerchantZIP to Text data type

```r
merchants$MerchantName <- as.character(merchants$MerchantName)
merchants$MerchantZIP <- as.character(merchants$MerchantZIP)
```

2.3.e Convert column MerchantCategoryCode to Categorical data type

```r
merchants$MerchantCategoryCode = factor(merchants$MerchantCategoryCode)
```

2.4 Create table for date

2.4.a Transform Time column to categorical (morning, afternoon, evening, night)

```r
# Convert column to Time data type
transactions_csv$Time <- as.POSIXct(strptime(transactions_csv$Time,"%H:%M"),"UTC")

# Define time range for each category
x=as.POSIXct(strptime(c("050000","105959","110000","155959","160000",
                        "185959"),"%H%M%S"),"UTC")

# Transform Time column
transactions_csv$Time = case_when(
  between(transactions_csv$Time,x[1],x[2]) ~"morning",
  between(transactions_csv$Time,x[3],x[4]) ~"afternoon",
  between(transactions_csv$Time,x[5],x[6]) ~"evening",
  TRUE ~"night")

head(transactions_csv$Time)
```

```
## [1] "morning"   "morning"   "morning"   "evening"   "morning"   "afternoon"
```

2.4.b Extract Date related column from the Transactions table

```r
date_temp <- data.frame(DateYear = transactions_csv$Year,
                        DateMonth = transactions_csv$Month,
                        DateDay = transactions_csv$Day,
                        Time = transactions_csv$Time
)
```

2.4.c Remove redundant data from the Date table

```
date_temp <- date_temp %>% distinct()
```

2.4.d Generate and add DateId column

```
date <- data.frame(DateId = 1:nrow(date_temp),
                        DateYear = date_temp$DateYear,
                        DateMonth = date_temp$DateMonth,
                        DateDay = date_temp$DateDay,
                        Time = date_temp$Time
)
```

2.4.e Convert column Time to Categorical data type

```
date$Time = factor(date$Time)
```

2.5 Fact table - transactions

2.5.a Replace existing merchant columns with single merchant id

```
# Uses Merchant Name (unique column in merchants table)
# to assign Merchant Id values in Transactions table
transactions_csv$MerchantId <- transactions_csv$Merchant.Name
conditions <- merchants$MerchantName
replacement_values <- merchants$MerchantId
transactions_csv$MerchantId <- replace(transactions_csv$MerchantId,
                                    transactions_csv$MerchantId %in% conditions,
                                    replacement_values)
```

2.5.b Replace existing date columns with single date id

```
# Combine all columns in Dates table
temp_dates = data.frame(DateId = date$DateId,
                        Condition = paste(date$DateYear,
                                          date$DateMonth,
                                          date$DateDay,
                                          date$Time)
)

# Combine all columns that were added to Date table from Transaction table
transactions_csv$Condition <- paste(transactions_csv$Year,
                                    transactions_csv$Month,
                                    transactions_csv$Day,
                                    transactions_csv$Time)
```

```r
# Add DateId column with temporary values
transactions_csv$DateId <- 1:nrow(transactions_csv)

# Replace DateId based on the combined Date columns in both tables
transactions_csv$DateId <- replace(transactions_csv$DateID,
                              transactions_csv$Condition %in% temp_dates$Condition,
                              temp_dates$DateId)
```

2.5.c Remove duplicates

```r
transactions_csv <- transactions_csv %>% distinct()
```

2.5.d Add TransactionId column

```r
transactions_csv$TransactionId <- 1:nrow(transactions_csv)
```

2.5.e Combine user id and card index to generate CardId column

2.5.f Rename column names

```r
transactions <- data.frame(TransactionId = transactions_csv$TransactionId,
                              CardId = paste(transactions_csv$User,
                                             transactions_csv$Card, sep = "$"),
                              UserId = transactions_csv$User,
                              DateId = transactions_csv$DateId,
                              MerchantId = transactions_csv$MerchantId,
                              Amount = transactions_csv$Amount
)

head(transactions, 3)
```

```
##   TransactionId CardId UserId DateId MerchantId  Amount
## 1             1    0$0      0      1          1 $134.09
## 2             2    0$0      0      2          2  $38.48
## 3             3    0$0      0      3          3 $120.34
```

2.5.g Convert column Amount to numeric

```r
transactions$Amount <- as.numeric(sub('.', '', transactions$Amount))
```

3. Advanced Dimensional Modeling

3.1 Junk Dimension table - transaction_flags

3.1.a Create table

```
transaction_flags <- data.frame(TransactionId = transactions_csv$TransactionId,
                                UseChip = transactions_csv$Use.Chip,
                                IsFraud = transactions_csv$Is.Fraud
)
head(transaction_flags, 3)
```

```
##   TransactionId           UseChip IsFraud
## 1             1 Swipe Transaction      No
## 2             2 Swipe Transaction      No
## 3             3 Swipe Transaction      No
```

3.1.b Covert columns - UseChip, IsFraud to categorical data type

```
transaction_flags$UseChip <- factor(transaction_flags$UseChip)
transaction_flags$IsFraud <- factor(transaction_flags$IsFraud)
```

3.2 Bridge Table

The Transaction and Error Types have many-to-many relationship.

3.2.a Create bridge table by splitting multiple errors

```
nrow(transactions_csv)
```

```
## [1] 24386900
```

```
# Split the errors list for each transaction
transaction_error_bridge <- transactions_csv %>% separate_longer_delim(Errors., delim = ",")
# Rename column
transaction_error_bridge$ErrorDescription <- transaction_error_bridge$Errors.
nrow(transaction_error_bridge)
```

```
## [1] 24388620
```

3.2.b Create error dimension table with unique ErrorId

```r
errors <- data.frame(ErrorDescription=distinct(transaction_error_bridge,ErrorDescription))
# remove Error Description with empty string
errors <- errors %>% filter(ErrorDescription != "")
# add id column
errors$ErrorId <- 1:length(errors$ErrorDescription)
head(errors)
```

```
##       ErrorDescription ErrorId
## 1     Technical Glitch       1
## 2 Insufficient Balance       2
## 3              Bad PIN       3
## 4       Bad Expiration       4
## 5      Bad Card Number       5
## 6              Bad CVV       6
```

3.2.c Add ErrorId to bridge table

```r
transaction_error_bridge <- transaction_error_bridge %>%
  left_join(errors, by = "ErrorDescription") %>%
  select(TransactionId, ErrorId)
```

4. Data Quality Assessment

4.1 Identifying and Handling Missing values

Identify missing values in users table

```r
colSums(is.na(users))
```

```
##                 UserId               FullName             CurrentAge
##                      0                      0                      0
##          RetirementAge              BirthYear             BirthMonth
##                      0                      0                      0
##                 Gender               Address1               Address2
##                      0                      0                   1472
##                   City                  State                Zipcode
##                      0                      0                      0
##               Latitude              Longitude PerCapitaIncomeZipcode
##                      0                      0                      0
##           YearlyIncome              TotalDebt               FICOScore
##                      0                      0                      0
##         NumCreditCards
##                      0
```

Address2 has 1472 NA values

Identify missing values in cards table

```r
colSums(is.na(cards))
```

```
##             CardId          CardBrand           CardType         CardNumber
##                  0                  0                  0                  0
##            Expires                CVV            HasChip        CardsIssued
##                  0                  0                  0                  0
##        CreditLimit       AcctOpenDate YearPINLastChanged
##                  0                  0                  0
```

Identify missing values in merchants table

```r
colSums(is.na(merchants))
```

```
##           MerchantId         MerchantName         MerchantCity
##                    0                    0                    0
##        MerchantState          MerchantZIP MerchantCategoryCode
##                    0                 1267                    0
```

MerchantZIP has 1267 NA values

Identify missing values in date table

```r
colSums(is.na(date))
```

```
##    DateId  DateYear DateMonth   DateDay      Time
##         0         0         0         0         0
```

Identify missing values in transactions table

```r
colSums(is.na(transactions))
```

```
## TransactionId        CardId        UserId        DateId    MerchantId
##             0             0             0             0             0
##        Amount
##             0
```

Identify missing values in transaction_flags table

```r
colSums(is.na(transaction_flags))
```

```
## TransactionId       UseChip       IsFraud
##             0             0             0
```

Handle missing values

1. Replace Address2 column with " " string, since it is usually an optional variable

```
users$Address2[is.na(users$Address2)] <- ""
```

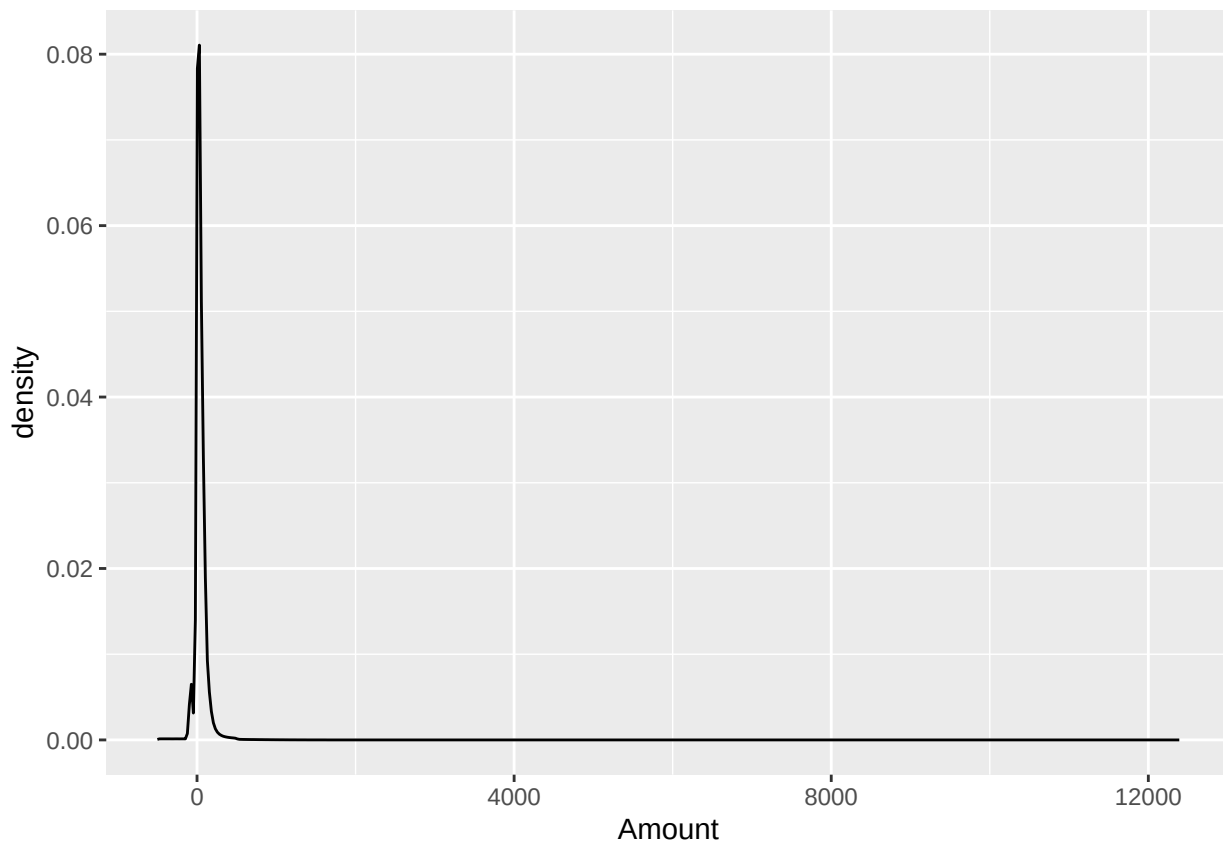
2. Drop rows where ZIP is NA, since it is an important feature for location based analysis of merchants and the count of rows with missing zip is very small(1267) when compared with the total number of rows(100,343) in the merchants table

```
merchants <- na.omit(merchants)
```

4.2 Detecting and Handling Outliers

4.2.a Detect outliers in Amount column in Transactions table

```
ggplot(transactions, aes(x=Amount)) + geom_density()
```




```
summary(transactions$Amount)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##  -500.00     9.20    30.14    43.63    65.06 12390.50
```

Handle outliers

Since, the number of outliers (Amount>500) is large, setting them to the value of upper bound calculated using IQR method would introduce skewness.

Better approach would be to transform the column. Group outliers into categories "Low", "Medium", "High", "Extreme"

```
transactions <- transactions %>%
  mutate(AmountCategory = case_when(
    Amount <= 150 ~ "Low",
    Amount <= 500 ~ "Medium",
    Amount <= 2000 ~ "High",
    TRUE ~ "Extreme"
  ))
```

4.2.b Detect outliers in CurrentAge column in Users table

```
Q1 <- quantile(users$CurrentAge, 0.25)
Q3 <- quantile(users$CurrentAge, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(users$CurrentAge)$out
```

```
## [1] 101
```

Handle ouliers

```
users$CurrentAge[users$CurrentAge < lower_bound] <- lower_bound
users$CurrentAge[users$CurrentAge > upper_bound] <- upper_bound
```

4.2.c Detect outliers in RetirementAge column in Users table

```
Q1 <- quantile(users$RetirementAge, 0.25)
Q3 <- quantile(users$RetirementAge, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(users$RetirementAge)$out
```

```
##   [1] 60 57 58 55 75 52 59 73 57 59 59 58 57 58 73 73 56 57 74 58 54 73 55 58 73
##  [26] 59 60 60 60 59 75 59 57 78 60 60 74 59 59 60 60 59 60 75 75 60 73 60 74 60
##  [51] 60 73 59 57 58 56 73 59 73 75 73 60 60 73 55 79 60 59 53 73 56 75 53 56 75
##  [76] 75 73 76 60 73 74 58 73 74 58 59 59 73 74 58 60 73 59 60 59 60 73 74 55 50
## [101] 73 73 73 57 59 57 75 60 75 59 50 73 59 57 54 73 58 60 55 60 57 57 60 54 58
## [126] 59 58 58 74 73 78 58 60 59 60 54 60 58 74 74 73 59 73 60 77 60 60 54 73 56
## [151] 75 74 74 73 55 57 57 74 58 60 60 59 59 73 55 55 60 73 74 57 75 60 57 57 59
## [176] 57 59 57 73 60 58 60 57 56 60 74 60 58 75 73 74 60 74 60 58 60 74 60 53 75
## [201] 59 60 58 74 73 60 60 60 60 60
```

Handle ouliers

```
users$RetirementAge[users$RetirementAge < lower_bound] <- lower_bound
users$RetirementAge[users$RetirementAge > upper_bound] <- upper_bound
```

4.2.d Detect outliers in PerCapitaIncomeZipcode column in Users table

```
Q1 <- quantile(users$PerCapitaIncomeZipcode, 0.25)
Q3 <- quantile(users$PerCapitaIncomeZipcode, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(users$PerCapitaIncomeZipcode)$out
```

```
##   [1] 163145  53797 106305  51642  95039  96516  49458  41750  40530  47055
##  [11]  52813  46762  51751  49477  42234  40694  46827  49629  51976      0
##  [21]  49868  47364  46169  51692  56720  42685  51032  56632  42028  48986
##  [31]  44540  79100 137428  49534  92938  58297  46616  51631  43432  49195
##  [41]  50607  50579  55274      0  45596  42685  94302  47798      0  56069
##  [51]      0      0  51692  47637  41682  49546  56252  45509  45818  42234
##  [61]  49629  44664  42850  47408  44196      0  63159  51032  52066  46461
##  [71]  60593  45685  47453      0  44747      0  43725  42884  47991      0
##  [81]      0  45132  41380  58517  62840  47524  42160  43827      0  48994
##  [91]  42884      0  44106  91487  45812  47698  53676      0  40761  41663
## [101]  55362  62177  50208  74205      0  40694      0  76725  45143  95039
## [111]  91180  41665  42201 150583  41979  45307  46175  69236  53790  55814
## [121]  75378  52517  46232
```

Handle ouliers

```
users$PerCapitaIncomeZipcode[users$PerCapitaIncomeZipcode < lower_bound] <- lower_bound
users$PerCapitaIncomeZipcode[users$PerCapitaIncomeZipcode > upper_bound] <- upper_bound
```

4.2.e Detect outliers in YearlyIncome column in Users table

```r
Q1 <- quantile(users$YearlyIncome, 0.25)
Q3 <- quantile(users$YearlyIncome, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(users$YearlyIncome)$out
```

```
##   [1] 249925 109687 216740  84694 193773 196784 100837  85128  82637  95945
##  [11] 107683  95348 105515 100880  82974 104692 101191 105963   2466 101679
##  [21]  96574  94133  88432  87030 104049 115465  85694  99883  90812 161276
##  [31] 280199 100991 189490 118862  95045  88554 100303 103185 103126 112695
##  [41]    553  92960  87030 192269  97455   2026 114318    920      1  97127
##  [51]  84987 101018 114692  92785  93417  86113 101193  87372  96661  90104
##  [61]   2422 128775 104045 106159  94733 123540 110570  96744   2370  91237
##  [71]   1785  89152  87436  99825  97845   1426  87317   2365  92017  84365
##  [81] 119308 128123  96901  85954  92368      3 103294  87437    645 186534
##  [91]  93406  97248 109440      4  83104  84944 112875 126775 102369 162709
## [101]      2  82972    399  92046  87659 193768 185909  86042 307018  85596
## [111]  92375 113514 141161 109673 113797 153691 107075  94260
```

Handle ouliers

```r
users$YearlyIncome[users$YearlyIncome < lower_bound] <- lower_bound
users$YearlyIncome[users$YearlyIncome > upper_bound] <- upper_bound
```

4.2.f Detect outliers in TotalDebt column in Users table

```r
Q1 <- quantile(users$TotalDebt, 0.25)
Q3 <- quantile(users$TotalDebt, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(users$TotalDebt)$out
```

```
##  [1] 191349 202328 241571 198417 437533 222735 225017 192458 210445 290730
## [11] 307856 265319 189899 247623 195657 241312 317964 211919 448929 189348
## [21] 197102 276156 201796 206422 219213 328089 210206 206000 206131 220951
## [31] 232506 236393 231619 255288 193215 199706 204251 240352 207423 233746
## [41] 197100 222502 200793 189558 200435 461854 516263 252106 242379 203172
## [51] 197377
```

Handle ouliers

```
users$TotalDebt[users$TotalDebt < lower_bound] <- lower_bound
users$TotalDebt[users$TotalDebt > upper_bound] <- upper_bound
```

4.2.g Detect outliers in CreditLimit column in cards table

```
Q1 <- quantile(cards$CreditLimit, 0.25)
Q3 <- quantile(cards$CreditLimit, 0.75)
IQR <- Q3 - Q1
# Calculate upper and lower bound
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

# Identify outliers
boxplot.stats(cards$CreditLimit)$out
```

```
##   [1]  46414  98100 132439 125723  68400  77237  53549  53036  53466  96907
##  [11]  43900  50915  39818  42116  39312  48502  40628  51012  47372  62081
##  [21]  39533  38342  51836  52609  45377  46201  40073  52585  52179  47709
##  [31]  54128  39389  38171  61324  43169  48370  58709  47359  41205  39578
##  [41]  40984  51302  38317  55693  40710  61355  45782  52121  47833  38648
##  [51]  41051  40350  47523  48003  39441  41816  39457  88743  57200 151223
##  [61]  61100  42142  82677  72868  38235  39764  40176  51504  86262  92939
##  [71]  64613  38290  41221  43033  43289  41057  43673  43973  60610  50961
##  [81]  42241  44568  65185  49425  47330  50774  75640  65540  42537  56640
##  [91]  39553  42218  39431  38084  62760  98334 141391  78790  39751  48344
## [101]  42491  62861  61399  39192  37722  76343  46070  39606  44345  37491
## [111]  40384  55750  46184  39114  42074  54768  43863  46621  62855  46521
## [121]  45854  38194  51051  48754  62317  38354  49107  37661  64059  49790
## [131]  45222  49705  43879  37941  37600  79717  47131  50127  42670  50521
## [141]  45141  39080  39431  77778  58311  66779  44314  43767  38385  37778
## [151]  37656  41713  51133  39632  48489  41040  39850  46342  37468  38837
## [161]  51951  39870  66930  72274 130971  52717  42541  39598  61407  40019
## [171]  64321  43186  56434  44885  38120  43122  40920  44390  37634  54842
## [181]  37377  41084  66781  47684  49679  57294  37400  43984  38814  41714
## [191]  66287  54517  62703  58234  76966  39591  39452  38048  38154  44717
## [201]  47153  70187  96637  44200  62519  87408  39422  38025 137669  58100
## [211]  49300  92828  55300  40914  39491  97352  54057  64565  61262  89900
## [221]  44100  45373  43287  42176  38900  51900  47211  37619  42253  51412
## [231]  37620  47183  44912  56039
```

Handle ouliers

```
cards$CreditLimit[cards$CreditLimit < lower_bound] <- lower_bound
cards$CreditLimit[cards$CreditLimit > upper_bound] <- upper_bound
```

5. Custom Functions in R

5.1 Standardize numeric variables to ensure consistent range across features

```
users$CurrentAgeScaled <- scale(users$CurrentAge)
users$RetirementAgeScaled <- scale(users$RetirementAge)
users$PerCapitaIncomeZipcodeScaled <- scale(users$PerCapitaIncomeZipcode)
users$YearlyIncomeScaled <- scale(users$YearlyIncome)
users$TotalDebtScaled <- scale(users$TotalDebt)
users$FICOScoreScaled <- scale(users$FICOScore)

cards$CreditLimitScaled <- scale(cards$CreditLimit)
```

5.2 Convert text columns to lowercase and trim whitespace

```
users <- users %>%  mutate(across(where(is.character), str_trim))
users <- users %>%  mutate(across(where(is.character), tolower))

merchants <- merchants %>%  mutate(across(where(is.character), str_trim))
merchants <- merchants %>%  mutate(across(where(is.character), tolower))
```

6. LOAD

Dimension table - users_dim

```
users_dim <- users %>% select(UserId, FullName, CurrentAgeScaled, RetirementAgeScaled,
                              BirthYear, BirthMonth, Gender, Address1, Address2,
                              City, State, Zipcode, Latitude, Longitude,
                              PerCapitaIncomeZipcodeScaled, YearlyIncomeScaled, TotalDebtScaled,
                              FICOScoreScaled, NumCreditCards) %>% distinct()
summary(users_dim)
```

```
##      UserId         FullName         CurrentAgeScaled.V1
##  Min.   :   1.0   Length:2000        Min.   :-1.4876231
##  1st Qu.: 500.8   Class :character   1st Qu.:-0.8358952
##  Median :1000.5   Mode  :character   Median :-0.0755461
##  Mean   :1000.5                      Mean   : 0.0000000
##  3rd Qu.:1500.2                      3rd Qu.: 0.6848030
##  Max.   :2000.0                      Max.   : 2.9658504
##  RetirementAgeScaled.V1   BirthYear      BirthMonth        Gender
##  Min.   :-1.8800617     Min.   :1918   Min.   : 1.000   Female:1016
##  1st Qu.:-0.4346017     1st Qu.:1961   1st Qu.: 3.000   Male  : 984
##  Median :-0.1133883     Median :1975   Median : 7.000
##  Mean   : 0.0000000     Mean   :1974   Mean   : 6.439
##  3rd Qu.: 0.5290384     3rd Qu.:1989   3rd Qu.:10.000
##  Max.   : 1.9744985     Max.   :2002   Max.   :12.000
##    Address1           Address2             City              State
```

```
##  Length:2000        Length:2000        Length:2000        Length:2000
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##    Zipcode             Latitude       Longitude
##  Length:2000        Min.   :20.88   Min.   :-159.41
##  Class :character   1st Qu.:33.84   1st Qu.: -97.39
##  Mode  :character   Median :38.25   Median : -86.44
##                     Mean   :37.39   Mean   : -91.55
##                     3rd Qu.:41.20   3rd Qu.: -80.13
##                     Max.   :61.20   Max.   : -68.67
##  PerCapitaIncomeZipcodeScaled.V1 YearlyIncomeScaled.V1  TotalDebtScaled.V1
##  Min.   :-2.5359480              Min.   :-2.4983805    Min.   :-1.3419135
##  1st Qu.:-0.7089714              1st Qu.:-0.6874776    1st Qu.:-0.8249278
##  Median :-0.2253950              Median :-0.2061490    Median :-0.0864315
##  Mean   : 0.0000000              Mean   : 0.0000000    Mean   : 0.0000000
##  3rd Qu.: 0.5090130              3rd Qu.: 0.5197911    3rd Qu.: 0.5778203
##  Max.   : 2.3359895              Max.   : 2.3306940    Max.   : 2.6819423
##  FICOScoreScaled.V1  NumCreditCards
##  Min.   :-3.417552   Min.   :1.000
##  1st Qu.:-0.427457   1st Qu.:2.000
##  Median : 0.026264   Median :3.000
##  Mean   : 0.000000   Mean   :3.073
##  3rd Qu.: 0.643622   3rd Qu.:4.000
##  Max.   : 2.086603   Max.   :9.000
```

```r
head(users_dim,4)
```

```
##   UserId       FullName CurrentAgeScaled RetirementAgeScaled BirthYear
## 1      1 hazel robinson        0.4132497          -0.1133883      1966
## 2      2     sasha sadr        0.4132497           0.5290384      1966
## 3      3     saanvi lee        1.9339480           0.2078250      1938
## 4      4  everlee clark        0.9563563          -1.0770284      1957
##   BirthMonth Gender               Address1 Address2        City State Zipcode
## 1         11 Female          462 rose lane             la verne    ca   91750
## 2         12 Female 3606 federal boulevard          little neck    ny   11363
## 3         11 Female        766 third drive          west covina    ca   91792
## 4          1 Female       3 madison street             new york    ny   10069
##   Latitude Longitude PerCapitaIncomeZipcodeScaled YearlyIncomeScaled
## 1    34.15   -117.76                   0.89417486          0.9447338
## 2    40.76    -73.74                   2.00293135          2.0109925
## 3    34.02   -117.89                   0.04493922         -0.6471240
## 4    40.71    -73.99                   2.33598954          2.3306940
##   TotalDebtScaled FICOScoreScaled NumCreditCards
## 1        1.408526       1.1494088              5
## 2        2.681942      -0.1299352              5
## 3       -1.337689      -0.1745635              5
## 4        2.681942       0.1824627              4
```

Dimension table - cards_dim

```
cards_dim <- cards %>% select(CardId, CardBrand, CardType, CardNumber, Expires,
                              CVV, HasChip, CardsIssued, CreditLimitScaled,
                              AcctOpenDate, YearPINLastChanged) %>% distinct()
summary(cards_dim)
```

```
##     CardId               CardBrand               CardType     CardNumber
##  Length:6146        Amex      : 402   Credit          :2057   Length:6146
##  Class :character   Discover  : 209   Debit           :3511   Class :character
##  Mode  :character   Mastercard:3209   Debit (Prepaid): 578   Mode  :character
##                     Visa      :2326
##
##
##     Expires               CVV            HasChip     CardsIssued
##  Min.   :1997-07-31   Length:6146        NO : 646   Min.   :1.000
##  1st Qu.:2020-02-29   Class :character   YES:5500   1st Qu.:1.000
##  Median :2021-09-30   Mode  :character              Median :1.000
##  Mean   :2020-11-06                                 Mean   :1.503
##  3rd Qu.:2023-05-31                                 3rd Qu.:2.000
##  Max.   :2024-12-31                                 Max.   :3.000
##  CreditLimitScaled.V1  AcctOpenDate        YearPINLastChanged
##  Min.   :-1.4242799   Min.   :1991-01-01   Min.   :2002
##  1st Qu.:-0.6950430   1st Qu.:2006-10-01   1st Qu.:2010
##  Median :-0.1203979   Median :2010-02-15   Median :2013
##  Mean   : 0.0000000   Mean   :2011-01-15   Mean   :2013
##  3rd Qu.: 0.5592671   3rd Qu.:2016-05-01   3rd Qu.:2017
##  Max.   : 2.4407323   Max.   :2020-02-01   Max.   :2020
```

```
head(cards_dim,4)
```

```
##   CardId CardBrand CardType       CardNumber    Expires CVV HasChip CardsIssued
## 1    0$0      Visa    Debit 4344676511950444 2022-12-31 623     YES           2
## 2    0$1      Visa    Debit 4956965974959986 2020-12-31 393     YES           2
## 3    0$2      Visa    Debit 4582313478255491 2024-02-29 719     YES           2
## 4    0$3      Visa   Credit 4879494103069057 2024-08-31 693      NO           1
##   CreditLimitScaled AcctOpenDate YearPINLastChanged
## 1         1.0913296   2002-09-01               2008
## 2         0.8503820   2014-04-01               2014
## 3         2.4407323   2003-07-01               2004
## 4        -0.1403302   2003-01-01               2012
```

Dimension table - merchants_dim

```
merchants_dim <- merchants %>% select(MerchantId, MerchantName, MerchantCity,
                                      MerchantState, MerchantZIP,
                                      MerchantCategoryCode) %>% distinct()
summary(merchants_dim)
```

```
##    MerchantId     MerchantName       MerchantCity       MerchantState
##  Min.   :     1   Length:99076       Length:99076       Length:99076
##  1st Qu.: 25570   Class :character   Class :character   Class :character
##  Median : 50536   Mode  :character   Mode  :character   Mode  :character
##  Mean   : 50475
##  3rd Qu.: 75466
##  Max.   :100343
##
##  MerchantZIP        MerchantCategoryCode
##  Length:99076       5411   : 9937
##  Class :character   5812   : 7814
##  Mode  :character   5912   : 6047
##                     5813   : 5064
##                     4900   : 4682
##                     5921   : 4617
##                     (Other):60915
```

```r
head(merchants_dim,4)
```

```
##   MerchantId        MerchantName  MerchantCity MerchantState MerchantZIP
## 1          1 3527213246127877120      la verne            ca       91750
## 2          2 -727612092139916032 monterey park            ca       91754
## 3          3 3414527459579106816 monterey park            ca       91754
## 4          4 5817218446178736128      la verne            ca       91750
##   MerchantCategoryCode
## 1                 5300
## 2                 5411
## 3                 5651
## 4                 5912
```

Dimension table - date_dim

```r
date_dim <- date %>% select(DateId, DateYear, DateMonth,
                            DateDay, Time) %>% distinct()
summary(date_dim)
```

```
##      DateId         DateYear      DateMonth         DateDay
##  Min.   :    1   Min.   :1991   Min.   : 1.000   Min.   : 1.00
##  1st Qu.:10366   1st Qu.:1998   1st Qu.: 3.000   1st Qu.: 8.00
##  Median :20731   Median :2005   Median : 7.000   Median :16.00
##  Mean   :20731   Mean   :2005   Mean   : 6.514   Mean   :15.73
##  3rd Qu.:31096   3rd Qu.:2013   3rd Qu.:10.000   3rd Qu.:23.00
##  Max.   :41461   Max.   :2020   Max.   :12.000   Max.   :31.00
##         Time
##  afternoon:10554
##  evening  :10374
##  morning  :10431
##  night    :10102
##
##
```

Dimension table - errors_dim

```
errors_dim <- errors %>% select(ErrorId, ErrorDescription) %>% distinct()
summary(errors_dim)
```

```
##     ErrorId    ErrorDescription
##  Min.   :1.0   Length:7
##  1st Qu.:2.5   Class :character
##  Median :4.0   Mode  :character
##  Mean   :4.0
##  3rd Qu.:5.5
##  Max.   :7.0
```

Bridge table - transaction_error_bridge

```
transaction_error_bridge <- transaction_error_bridge %>%
  select(TransactionId, ErrorId) %>% distinct()
summary(transaction_error_bridge)
```

```
##  TransactionId         ErrorId
##  Min.   :       1   Min.   :1
##  1st Qu.: 6096725   1st Qu.:2
##  Median :12193476   Median :2
##  Mean   :12193460   Mean   :2
##  3rd Qu.:18290180   3rd Qu.:2
##  Max.   :24386900   Max.   :7
##                     NA's   :23998469
```

Junk Dimension table - transaction_flags

```
transaction_flags <- transaction_flags %>% select(TransactionId, UseChip, IsFraud)
summary(transaction_flags)
```

```
##  TransactionId                    UseChip         IsFraud
##  Min.   :       1   Chip Transaction  : 6287598   No :24357143
##  1st Qu.: 6096726   Online Transaction: 2713220   Yes:   29757
##  Median :12193450   Swipe Transaction :15386082
##  Mean   :12193450
##  3rd Qu.:18290175
##  Max.   :24386900
```

Fact table - transactions_fact

```
transactions_fact <- transactions %>% select(TransactionId, UserId, CardId,
                                             DateId, AmountCategory
                                             ) %>% distinct()
summary(transactions_fact)
```

```
##  TransactionId          UserId        CardId              DateId
##  Min.   :       1   Min.   :   0   Length:24386900    Min.   :    1
##  1st Qu.: 6096726   1st Qu.: 510   Class :character   1st Qu.:10356
##  Median :12193450   Median :1006   Mode  :character   Median :20724
##  Mean   :12193450   Mean   :1001                      Mean   :20726
##  3rd Qu.:18290175   3rd Qu.:1477                      3rd Qu.:31093
##  Max.   :24386900   Max.   :1999                      Max.   :41461
##  AmountCategory
##  Length:24386900
##  Class :character
##  Mode  :character
##
##
##
```